Improved Cryptanalysis of FEA-1 and FEA-2 using Square Attacks

Amit Kumar Chauhan[1], Abhishek Kumar[2], Somitra Kumar Sanadhya[3]

[1] QNu Labs Private Limited, Bengaluru, India
akcindia.macs@gmail.com
[2] ZF India Private Limited, Hyderabad, India
abhishek.kumar.iitrpr@gmail.com
[3] Indian Institute of Technology Jodhpur, Jodhpur, India
somitra@iitj.ac.in

Abstract. This paper presents a security analysis of the South Korean Format-Preserving Encryption (FPE) standards FEA-1 and FEA-2. In 2023, Chauhan *et al.* presented the first third-party analysis of FEA-1 and FEA-2 against the square attack. The authors proposed new distinguishing attacks covering up to three rounds of FEA-1 and five rounds of FEA-2, with a data complexity of 2^8 plaintexts. Additionally, using these distinguishers, they presented key recovery attacks for four rounds of FEA-1 and six rounds of FEA-2, for 192-bit and 256-bit key sizes. The complexities of both the four-round FEA-1 and six-round FEA-2 key recovery attacks are $2^{137.6}$.

In this work, we successfully extend the number of rounds attacked for both FEA-1 and FEA-2, using the square attack technique. Specifically, we present a four-round distinguishing attack against FEA-1 and six-round distinguishing attack against FEA-2. The data complexities of these distinguishers are 2^{64} plaintexts. Furthermore, we apply these distinguishers to perform key recovery attacks on five rounds of FEA-1 and seven rounds of FEA-2, targeting the 256-bit key size. The time complexities of the presented key recovery attacks are $2^{193.6}$.

Keywords: Format Preserving Encryption, FEA-1, FEA-2, Square attack, Round Function.

1 Introduction

Format-Preserving Encryption (FPE) is a cryptographic primitive that transforms data from a predefined domain into itself. Specifically, FPE refers to transformation of data that is formatted as a sequence of the symbols in such a way that the encrypted form of the data has the same format and length as the original data. A common application of FPE is the encryption of Social Security numbers (SSNs), where the ciphertext remains a valid SSN. Other significant applications of FPE include the encryption of credit card numbers, healthcare records, and other forms of personally identifiable information (PII). Essentially,

FPE allows for the encryption of sequences of digits or letters while preserving their original length and format.

Commonly used encryption methods such as block ciphers transform a message into corresponding ciphertext to ensure its confidentiality. These block ciphers such as PRESENT [4] and AES [12], treat plaintexts as a stream of bits. The domain sizes of these block ciphers are fixed-size binary strings, disregarding any specific format that the data might have. In practice, it is highly unlikely that the length of input message and the input size of the block cipher are equal. To cater encryption of data of size, non equal to the block size, NIST has suggested various modes of operations [16]. In many real life applications, it is essential to have the ciphertext follow the same format as the plaintext, without ciphertext length expansion, considering both efficiency and regulatory aspects. Since conventional block ciphers cannot fulfill this requirement, FPE schemes are employed in many industries, especially financial and healthcare sectors. These sectors utilize FPE to comply with stringent data security regulations while ensuring compatibility with legacy systems that store and process data in specific formats.

1.1 Related Work

Brightwell and Smith [7], from the database community, were the first to recognize the issue of encryption over fixed formats. They noted that when a traditional block cipher is used to encrypt plaintext in a specific format, it produces ciphertext that "bears roughly the same resemblance to plaintext... as a hamburger does to a T-bone steak". In 2002, Black and Rogaway [3] systematically addressed the problem of designing FPE schemes and proposed three different methods for achieving FPE. They introduced Feistel constructions and explored the applicability of the cycle-walking technique in designing FPE schemes, referring to this third method as the 'Generalized-Feistel Cipher'. In 2009, Bellare *et al.* [1] presented the FFX construction, which is based on Feistel constructions. FFX stands for *Format-preserving, Feistel-based encryption*, with 'X' indicating multiple instantiations of the proposed construction. Notably, many popular FPE schemes, including the NIST standards FF1 and FF3-1, follow the design principle of combining cycle walking with Feistel networks [6, 17, 24]. Considering the efficiency issue of Feistel-based constructions, there are alternative approaches that use substitution-permutation network (SPN)-based FPE constructions [8, 9, 15, 19].

Considering efficiency and flexibility aspects, Feistel [18] and Generalized-Feistel [5, 22] structures are among the most extensively used and well-analyzed cryptographic designs. In 2014, Lee *et al.* [21] proposed two new Feistel-based FPE schemes, FEA-1 and FEA-2. Similar to the FFX construction, FEA-1 and FEA-2 are families of format-preserving encryption schemes based on the Feistel structure. These schemes are currently the South Korean FPE standards (TTAK.KO-12.0275). A notable distinction between FFX and FEA-1/2 is that the latter employs a dedicated tweakable round function instead of AES-128. FEA-1 and FEA-2 support key lengths of 128,

192, and 256 bits. According to the designers, FEA-1 and FEA-2 are nearly twice as efficient as FF1 and FF3-1.

In 2020, Dunkelman *et al.* [14] presented full-round distinguishing attacks for all versions of FEA-1 and FEA-2. They also demonstrated full-round key-recovery attacks on FEA-1 and FEA-2 with 192-bit and 256-bit key variants. Notably, these attacks apply only to the smallest domain size of these schemes, specifically the 8-bit case. For a domain size of N^2 and number of rounds r, the time and data complexity of these attacks are $O(N^{r-3})$ and $O(N^{r-4})$, respectively. In 2021, Tim Beyne [2] analyzed the security of FEA-1 and FEA-2 using the linear cryptanalysis technique [23]. According to this analysis, the time and data complexities for both distinguishing and message-recovery attacks on FEA-1 are $O(N^{r/2-1.5})$. For FEA-2, the time and data complexities for distinguishing and message-recovery attacks are $O(N^{r/3-1.5})$ and $O(N^{r/3-0.5})$, respectively.

Interestingly, the designers of FEA-1/2 [21] claimed the security against the square attacks [11] by stating: "*Square attacks are not effective against the TBCs since the KSP-KSP function with MDS diffusion layers does not admit good integral characteristics. There are 3-round integral characteristics for our TBCs for some block bit lengths. But there do not seem to exist useful characteristics when the number of rounds is greater than 4*". However, in 2023, Chauhan *et al.* [10] presented first third-party analysis of FEA-1 and FEA-2 based on square attacks [11]. They proposed distinguisher that cover up to three rounds of FEA-1 and five rounds of FEA-2, for all key sizes 128-bit, 192-bit and 256-bit. The data and time complexity of these distinguishers are 2^8 plaintexts and encryptions, respectively. Using these square-attack based distinguishers, they also performed key recovery attacks for FEA-1 and FEA-2 with 192-bit and 256-bit key sizes. The complexities of these four round FEA-1 and six round FEA-2 key recovery attacks are $2^{137.6}$.

1.2 Our Contribution

In this work, we analyze the security of FEA-1 and FEA-2 and improve the best known square attacks. To be specific, we present distinguishing attacks for four rounds of FEA-1 and six rounds of FEA-2, applicable for all key sizes. The data and time complexity of the FEA-1 distinguisher are 2^{64} plaintexts and encryptions, respectively. Furthermore, we utilize the proposed distinguishers to mount key recovery attacks on both FEA-1 and FEA-2. These key recovery attacks cover up to five rounds of FEA-1 and seven rounds for FEA-2. The time complexity of the key recovery attack for both FEA-1 and FEA-2 is $2^{193.6}$. In Table 1, we include the results of the best known square attacks and summarize the results of the attacks presented in this work, for both FEA-1 and FEA-2.

This work reaffirms that square attacks do not present a significant threat to the ciphers under consideration, as the number of rounds successfully attacked is roughly one-third of the total rounds. Note that the attacks discussed in this paper are applicable only to a 128-bit block size. However, the attacks presented by Chauhan *et al.* [10] are effective for any block size of 16-bit or more.

Table 1. Summery of square attacks on reduced-round FEA-1 and FEA-2.

Algorithm	#Rounds	Key Size	Attack Type	Complexity	Ref.
FEA-1	3	128/192/256	Distinguisher	2^8	[10]
FEA-1	4	192/256	Key Recovery	$2^{137.6}$	[10]
FEA-1	4	128/192/256	Distinguisher	2^{64}	§ 4.1
FEA-1	5	256	Key Recovery	$2^{193.6}$	§ 5.1
FEA-2	5	128/192/256	Distinguisher	2^8	[10]
FEA-2	6	192/256	Key Recovery	$2^{137.6}$	[10]
FEA-2	6	128/192/256	Distinguisher	2^{64}	§ 4.2
FEA-2	7	256	Key Recovery	$2^{193.6}$	§ 5.2

1.3 Organization of the Paper

In Section 2, we explain the notations used throughout the paper, and describe the FEA construction, followed by a brief introduction to the square attack. Section 3 discusses the existing square attacks on FEA-1 and FEA-2. In Section 4, we propose a four-round distinguisher for FEA-1 and a six-round distinguisher for FEA-2, both based on the square attack strategy. Section 5 presents a five-round key recovery attack for FEA-1 and a seven-round key recovery attack for FEA-2. Finally, Section 6 concludes our work and outlines future research directions.

2 Preliminaries

2.1 Notations

Throughout the paper, we will use the following notations.

- $|x|$: Length of bit-string x.
- $X[i]$: $(i+1)^{\text{th}}$ byte of string X.
- $X_R^{(r-1)}$: Right half input of the r^{th} round.
- $X_L^{(r-1)}$: Left half input of the r^{th} round.
- $Rk_a^{(r)}$ and $Rk_b^{(r)}$: The r^{th} round subkeys.
- T_r : Tweak input for the r^{th} round
- C_L : Left half of the ciphertext
- C_R : Right half of the ciphertext.

2.2 Specification of FEA

FEA-1 and FEA-2 [21] are format-preserving encryption algorithms that employ a Feistel structure with tweakable round functions (TBC). Both FEA-1 and FEA-2, support domain sizes ranging from 2^8 to 2^{128}, The suggested number of rounds are 12 and 18 for 128-bit keys, respectively. FEA-1 and FEA-2, also support 192-bit and 256-bit keysizes. The number of rounds specified for each variant, based on the key bit lengths, is mentioned in Table 2.

Table 2. Number of rounds according to the key bit-lengths and TBC types.

Key Length	FEA-1	FEA-2
128	12	18
192	14	21
256	16	24

Both the FEA-1 and FEA-2 families consist of three main components: the key schedule, the tweak schedule, and the round function. In the following subsections, we provide a brief description of each of these components.

2.2.1 Round Function

The round function of FEA is composed of Tw-KSP-KSP-Tr functions as defined by Lee *et al.* [21]. Here, Tw and Tr define round tweak addition and truncation, respectively. The KSP function consists of round key addition (K), substitution layer (S) and diffusion layer (P). The substitution layer (S) comprises eight parallel identical 8-bit S-boxes, while the diffusion layer (P) is defined as multiplication with an 8×8 MDS matrix $\mathcal{M}$ defined over the field $\mathsf{GF}(2^8)$.

$$M = \begin{pmatrix} 28 & 1a & 7b & 78 & c3 & d0 & 42 & 40 \\ 1a & 7b & 78 & c3 & d0 & 42 & 40 & 28 \\ 7b & 78 & c3 & d0 & 42 & 40 & 28 & 1a \\ 78 & c3 & d0 & 42 & 40 & 28 & 1a & 7b \\ c3 & d0 & 42 & 40 & 28 & 1a & 7b & 78 \\ d0 & 42 & 40 & 28 & 1a & 7b & 78 & c3 \\ 42 & 40 & 28 & 1a & 7b & 78 & c3 & d0 \\ 40 & 28 & 1a & 7b & 78 & c3 & d0 & 42 \end{pmatrix}$$

The round function supports input/output bit-length in $[4, \cdots, 64]$. A complete description of the round function is depicted in Fig. 1.

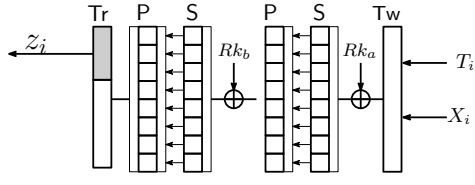

Fig. 1. Round function of FEA algorithm.

2.2.2 Key Schedule

FEA-1 and FEA-2 utilize the same key schedule, which takes the secret key K and the block size n as inputs. Each iteration of the key schedule produces four

64-bit round keys. During each round of encryption or decryption, two of these 64-bit sub-keys are used, meaning that the four round keys are employed across two consecutive rounds.

2.2.3 Tweak Schedule

The main difference between FEA-1 and FEA-2 is the tweak schedule. Let n denote the block bit-length of TBC. We define $n_1 = \lceil n/2 \rceil$ and $n_2 = \lfloor n/2 \rfloor$. The tweak schedule for each type TBC is described as follows.

For FEA-1, the tweak T of bit-length $(128-n)$ is divided into two sub-tweaks: T_L with length $64 - n_2$ and T_R with length $64 - n_1$. For every round, we set $T_a^i = 0$, and define T_b^i for the i-th round by

$$T_b^i = \begin{cases} T_L & \text{if } i \text{ is odd} \\ T_R & \text{if } i \text{ is even} \end{cases}$$

For FEA-2, the tweak T of bit-length 128 is divided into two sub-tweaks: T_L and T_R of 64 bits long. Here, T_L consists of the first 64 consecutive bits of T, while T_R comprises the next 64 consecutive bits. The values of T_a^i and T_b^i for the i-th round are then determined as follows:

$$T_a^i || T_b^i = \begin{cases} 0 & \text{if } i \equiv 1 \pmod 3 \\ T_L & \text{if } i \equiv 2 \pmod 3 \\ T_R & \text{if } i \equiv 0 \pmod 3 \end{cases}$$

2.3 Square Attack

The square attack, also known as the integral attack, is a cryptanalysis technique particularly effective against certain block ciphers, especially those utilizing a substitution-permutation network (SPN) structure. The key idea behind the square attack is to exploit the algebraic properties of the cipher's design, particularly the way inputs mix during the rounds of encryption. In the case of a byte-oriented cipher, the attack begins with a Λ-set, where some state bytes are all different and the remaining state bytes are equal. The bytes that vary are referred to as 'active' (A) bytes, and those that remain constant are called 'passive' (C) bytes. Additionally, a byte is classified as 'balanced' (B) if the sum of all 2^8 possible values is zero; conversely, if the sum cannot be predicted, it is labeled as an 'unknown' (?) byte.

The main operations used by FEA-1 and FEA-2 include substitution layer (S), diffusion layer (P), key addition layer (K), tweak addition, and xor ($\oplus$). When the substitution and key addition layers are applied to an active or passive byte, the output will likewise be an active or passive byte. Essentially, when a Λ-set is used as input for the substitution layer and key addition layer, it produces another Λ-set as output.

The permutation layer (P) of FEA-1 and FEA-2 utilizes an 8×8 MDS matrix, in which each output byte is a linear combination of all eight input bytes. When

the permutation layer is applied to an all-passive byte input, the output will also consist entirely of passive bytes. In contrast, if an input Λ-set contains a single active byte, the resulting output will have all bytes active.

The XOR operation between an active byte and a balanced byte results in a balanced byte. Additionally, XORing an active byte with another active byte will also yield a balanced byte. Furthermore, Table 3 illustrates the properties of the XOR operation.

Table 3. Properties of XOR operation. Here, A, B and C denotes active bytes, balanced bytes and passive bytes respectively [25].

$\oplus$	A	B	C
A	B	B	A
B	B	B	B
C	A	B	C

With respect to tweak addition, if the tweak is a constant value, the input active/passive byte will output an active/passive byte. However, the round tweak can be chosen as a Λ-set where one byte of the tweak is active. In these cases, if both the tweak byte and the state byte are active such that $\Lambda_i \oplus T_i = C$ for $0 \leq i \leq 255$ for each i, the particular byte will become a constant byte after the tweak addition [10, 13].

3 Existing Square Attacks on FEA Constructions

In this section, we revisit the existing distinguishers for FEA-1 and FEA-2 as presented in [10].

3.1 Three-Round Distinguisher for FEA-1

We briefly describe the 3-round distinguisher for FEA-1 as presented by Chauhan *et al.* [10]. For this attack, the block size n is 128-bits and the corresponding tweak length is 0-bit. Note that this distinguisher is valid for any block length of 16-bit or more, with the corresponding tweak sizes chosen as per design specifications.

As shown in Fig. 2, the attack begins by choosing a Λ-set of 2^8 plaintexts, where only the first byte is active and the remaining bytes are held constant. After executing three rounds of FEA-1, it can be observed that all the bytes of the left half of the third round output are balanced.

The three-round distinguisher starts with choosing plaintexts $X_L^{(0)}$ and $X_R^{(0)}$ which are as follows:

$$X_L^{(0)} = (A, \beta_2, \beta_3, \beta_4, \beta_5, \beta_6, \beta_7, \beta_8) \tag{1}$$

$$X_R^{(0)} = (\alpha_1, \alpha_2, \alpha_3, \alpha_4, \alpha_5, \alpha_6, \alpha_7, \alpha_8), \tag{2}$$

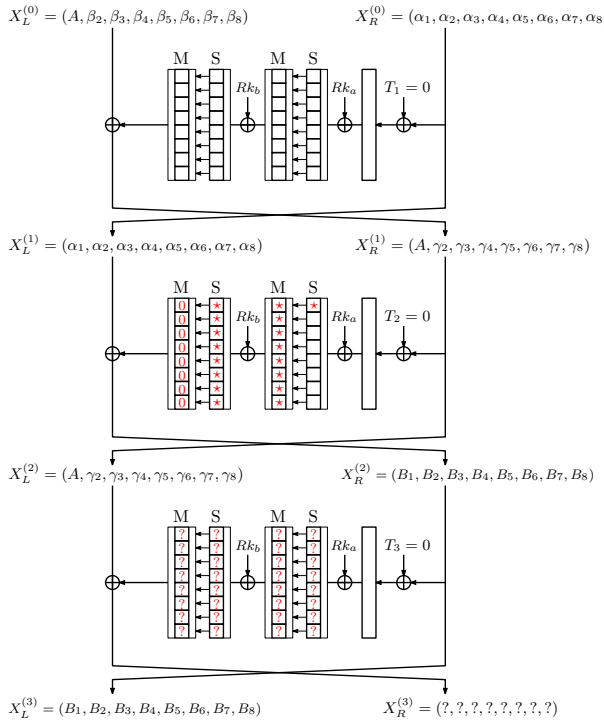

Fig. 2. A 3-round distinguisher for FEA-1.

where the byte A takes all 2^8 values (a Λ-set), and other bytes α_i for $1 \leq i \leq 8$ and β_j for $2 \leq j \leq 8$ are fixed arbitrary constants.

After executing the first round on inputs $(X_L^{(0)}, X_R^{(0)})$, the second round inputs $X_L^{(1)}$ and $X_R^{(1)}$ become:

$$X_L^{(1)} = (\alpha_1, \alpha_2, \alpha_3, \alpha_4, \alpha_5, \alpha_6, \alpha_7, \alpha_8) \tag{3}$$

$$X_R^{(1)} = (A, \gamma_2, \gamma_3, \gamma_4, \gamma_5, \gamma_6, \gamma_7, \gamma_8). \tag{4}$$

After applying the second round function to $X_R^{(1)}$, we expect that each byte of the right output of the second round is balanced.

$$X_L^{(2)} = (A, \gamma_2, \gamma_3, \gamma_4, \gamma_5, \gamma_6, \gamma_7, \gamma_8) \tag{5}$$

$$X_R^{(2)} = (B_1, B_2, B_3, B_4, B_5, B_6, B_7, B_8). \tag{6}$$

After applying the third round, and it can be easily observed that all the bytes of the left half of third round output are balanced.

3.2 Five-Round Distinguisher for FEA-2

As shown in Fig. 3, the 5-round distinguishing attack for FEA-2 begins by selecting a Λ-set of 2^8 plaintexts, where only the first byte is active and the remaining bytes are held constant. For simplicity, the authors [10] considered the block size for this attack to be 128 bits (i.e., $n = 128$). However, this attack is valid for all domain sizes of 16 bits or more. The tweak T is always a 128-bit value, and the generation of round tweaks is explained in Section 2.2. Additionally, this attack exploits the tweak scheduling algorithm, as detailed below. After running five rounds of FEA-2, it can be observed that all the bytes of the left half of the fifth round output are balanced.
This five round distinguishing attack on FEA-2 begins with a Λ-set of 256 plaintexts as input, where the first byte is active and all other bytes are arbitrary constants, i.e.,

$$X_L^{(0)} = (A, \beta_2, \beta_3, \beta_4, \beta_5, \beta_6, \beta_7, \beta_8) \tag{7}$$

$$X_R^{(0)} = (\alpha_1, \alpha_2, \alpha_3, \alpha_4, \alpha_5, \alpha_6, \alpha_7, \alpha_8) \tag{8}$$

After executing the first round, the first byte of $X_R^{(1)}$, i.e., $X_R^{(1)}[0]$ is active and remaining bytes are still constant values.

$$X_L^{(1)} = (\alpha_1, \alpha_2, \alpha_3, \alpha_4, \alpha_5, \alpha_6, \alpha_7, \alpha_8) \tag{9}$$

$$X_R^{(1)} = (A, \gamma_2, \gamma_3, \gamma_4, \gamma_5, \gamma_6, \gamma_7, \gamma_8) \tag{10}$$

In this attack, the round tweaks T_2 is selected such that the xor of the round tweaks and the corresponding round inputs remains a constant value. Since only the first byte of $X_R^{(1)}$ is an active byte, the corresponding tweak byte $T_2[0]$ (the first byte of T_2) is chosen to be active such that $(X_R^{(1)}[0]_i \oplus T_2[0]_i)$ is passive (constant value) for $0 \leq i \leq 255$. Consequently, the second round function is not active, meaning the left half of the output of the second round remains constant, and the input to the third round is also constant.

After extending this distinguisher for three additional rounds (Fig. 3), i.e., total five rounds, all the bytes of the left half of the fifth round output are balanced.

4 Improved Square Attacks on FEA Constructions

In this section, we present improved distinguishers for FEA-1 and FEA-2. We evaluate the security and resistance of the considered ciphers against square attacks. In order to improve the square attacks for more number of rounds, we treat the internal round functions F_j, for $0 \leq j \leq 3$, as 64-bit random permutations defined over $\{0,1\}^{64}$, rather than using the typical double-SP layer type of round functions. The internal round function F is depicted in Fig. 4.

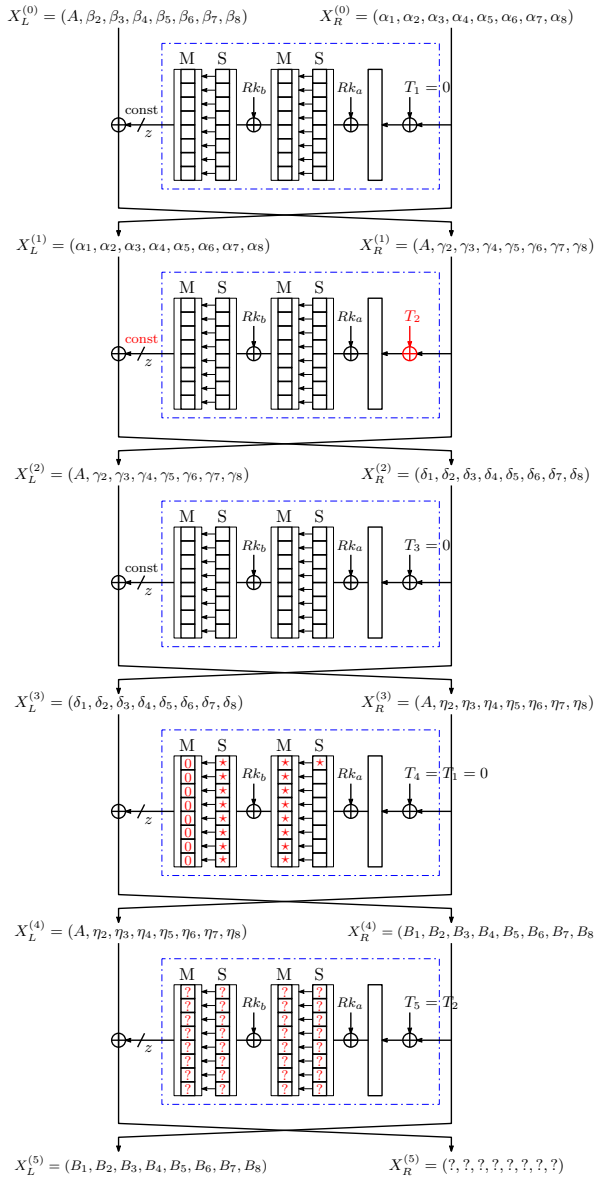

Fig. 3. A 5-round distinguisher for FEA-2.

Note 1. Since the substitution layer S and the permutation layer P used in FEA construction represent two different 64-bit permutations defined over $\{0,1\}^{64}$, thus their compositions will also be a permutation over $\{0,1\}^{64}$. As a result, by fixing the round keys, a cryptographic 64-bit permutation can be obtained via a SP layer. Furthermore, since keys are fixed for each permutation and a single

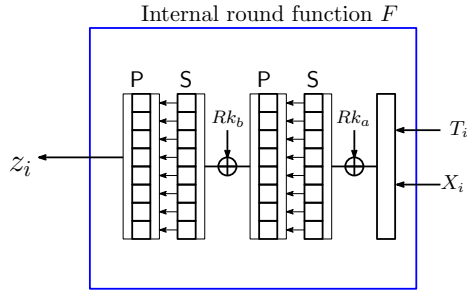

Fig. 4. Internal Round function of FEA construction, where F is considered as a 64-bit random permutation (a composition of two SP layers, with fix round keys).

SP layer is a permutation, so again, the composition of two different SP layers will also be a permutation. For example, Simpira cipher [20] uses "two rounds of AES" as a basic building block to construct an internal round function.

4.1 Improved Four-Round Distinguisher for FEA-1

We now present four-round distinguishers for FEA-1 (where the internal round function is treated as a random permutation) as shown in Fig. 5. In order to mount this distinguisher, we need 2^{64} number of four round encryptions. Therefore, the time and data complexity of this distinguisher are 2^{64}. The block size n chosen for this attack is 128-bit and the corresponding tweak size is 0-bit.

To initiate this attack, we define a set Γ of 2^{64} plaintexts, where the first eight bytes are active and remaining eight bytes are constants. After running four rounds of FEA-1, we observe that all bytes in the left half of the third-round output are balanced.

We now describe a 4-round distinguisher for FEA-1 which is as follows:

Let $X_L^{(r-1)}$ and $X_R^{(r-1)}$ be the right and left inputs of the state for the r^{th} round, respectively.

1. Choose a set Γ of 2^{64} plaintexts $\mathbf{P} = \{(X_{L,i}^{(0)}, X_{R,i}^{(0)} \mid 0 \leq i < 2^{64})$ such that all eight bytes of $X_L^{(0)}$ are active and all eight bytes of $X_R^{(0)}$ are constants.
2. Set round tweaks $T_1 = T_2 = T_3 = T_4 = 0$ by the design specifications of FEA-1.
3. For $0 \leq i < 2^{64}$:
   (a) For $0 \leq j \leq 3$:
      i. Compute $(X_{L,i}^{(j+1)}, X_{R,i}^{(j+1)})$ from $(X_{L,i}^{(j)}, X_{R,i}^{(j)})$ as:

$$(X_{L,i}^{(j+1)}, X_{R,i}^{(j+1)}) \leftarrow \mathbf{F}_{(j+1)}(X_{L,i}^{(j)}, X_{R,i}^{(j)}).$$

4. Check that all bytes of $X_L^{(4)}$ are balanced.

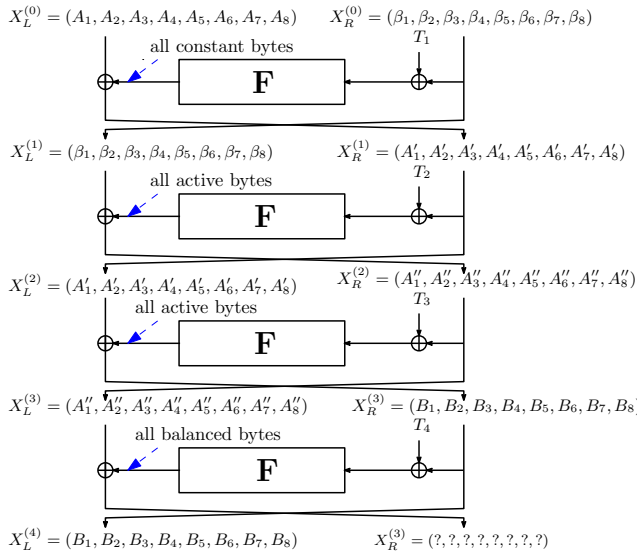

Fig. 5. Improved 4-round distinguisher for FEA-1.

We further describe the details of the four-round distinguisher attack procedure. Start with choosing plaintexts $X_L^{(0)}$ and $X_R^{(0)}$ as follows:

$$X_L^{(0)} = (A_1, A_2, A_3, A_4, A_5, A_6, A_7, A_8) \tag{11}$$

$$X_R^{(0)} = (\alpha_1, \alpha_2, \alpha_3, \alpha_4, \alpha_5, \alpha_6, \alpha_7, \alpha_8), \tag{12}$$

where each byte A_i for $1 \leq i \leq 8$ takes all 2^8 values to form a Λ-set, while all A_i's together form a Γ-set. Here, each α_i for $1 \leq i \leq 8$ are arbitary constants.

After executing the first round on inputs $(X_L^{(0)}, X_R^{(0)})$, the second round inputs $X_L^{(1)}$ and $X_R^{(1)}$ become

$$X_L^{(1)} = (\alpha_1, \alpha_2, \alpha_3, \alpha_4, \alpha_5, \alpha_6, \alpha_7, \alpha_8) \tag{13}$$

$$X_R^{(1)} = (A'_1, A'_2, A'_3, A'_4, A'_5, A'_6, A'_7, A'_8). \tag{14}$$

where each byte A'_i for $1 \leq i \leq 8$ takes all 2^8 values, forming a Λ-set.

After executing the second round on inputs $(X_L^{(1)}, X_R^{(1)})$, the third round inputs $X_L^{(2)}$ and $X_R^{(2)}$ become

$$X_L^{(2)} = (A'_1, A'_2, A'_3, A'_4, A'_5, A'_6, A'_7, A'_8) \tag{15}$$

$$X_R^{(2)} = (A''_1, A''_2, A''_3, A''_4, A''_5, A''_6, A''_7, A''_8). \tag{16}$$

where each byte A''_i for $1 \leq i \leq 8$ takes all 2^8 values, forming a Λ-set.

After executing the third round on inputs $(X_L^{(2)}, X_R^{(2)})$, the third round inputs $X_L^{(3)}$ and $X_R^{(3)}$ become

$$X_L^{(3)} = (B_1, B_2, B_3, B_4, B_5, B_6, B_7, B_8) \tag{17}$$

$$X_R^{(3)} = (A_1'', A_2'', A_3'', A_4'', A_5'', A_6'', A_7'', A_8''). \tag{18}$$

After applying the forth round function to $(X_L^{(3)}, X_R^{(3)})$, we expect that each byte right output $X_R^{(4)}$ of the forth round are balanced (refer to Fig. 5). The data complexity of this four-round distinguisher is 2^{64} plaintexts, and the time complexity is 2^{64} encryptions.

4.2 Improved Six-Round Distinguisher for FEA-2

We now present a six-round distinguisher for FEA-2 by exploiting the tweak scheduling algorithm, as shown in Fig. 6. The block size n chosen for this attack is 128-bit and the corresponding tweak size is 128-bit (as per the design specifications, the tweak size is always 128-bit for FEA-2).

We initiate with a set of 2^{64} plaintexts, where all sixteen bytes are constant. The 128-bit tweak T is chosen in such a way that the right-half T_R of tweak T forms a Γ set consisting of all possible values from $\{0,1\}^{64}$. After running six rounds of FEA-2, we observe that all bytes in the left half of the sixth-round output are balanced.

Let $X_L^{(r-1)}$ and $X_R^{(r-1)}$ be the right and left inputs of the state for the r^{th} round, respectively. We now describe a 6-round distinguisher for FEA-2, which is as follows:

1. Choose a set of 2^{64} plaintexts $\mathbf{P} = \{(X_{L,i}^{(0)}, X_{R,i}^{(0)} \mid 0 \leq i < 2^{64})$ such that all eight bytes of X_L^0 and all eight bytes of X_R^0 are constants.
2. Set 64-bit round tweak $T_1 = T_2 = 0$ (by design specification $T_4 = T_5 = 0$) and $T_3 = T_6 = \Gamma$, i.e., all eight bytes of T_3 and T_6 are active.
3. For $0 \leq i < 2^{64}$:
   (a) Compute $(X_{L,i}^{(2)}, X_{R,i}^{(2)})$ from $(X_{L,i}^{(0)}, X_{R,i}^{(0)}, T_1)$ (by applying two round of FEA-2):

$$(X_{L,i}^{(1)}, X_{R,i}^{(1)}) \leftarrow \mathbf{F}_1(X_{L,i}^{(0)}, X_{R,i}^{(0)}, T_1).$$

   and

$$(X_{L,i}^{(2)}, X_{R,i}^{(2)}) \leftarrow \mathbf{F}_2(X_{L,i}^{(1)}, X_{R,i}^{(1)}, T_2).$$

   (b) Choose T_3 such that $(X_{R,l}^{(1)} \oplus T_{3,l})$ are constants for $0 \leq l < 2^{64}$.
   (c) For $2 \leq j \leq 5$:
      i. Compute $(X_{L,i}^{(j+1)}, X_{R,i}^{(j+1)})$ from $(X_{L,i}^{(j)}, X_{R,i}^{(j)}, T_{(j+1)})$:

$$(X_{L,i}^{(j+1)}, X_{R,i}^{(j+1)}) \leftarrow \mathbf{F}_{(j+1)}(X_{L,i}^{(j)}, X_{R,i}^{(j)}, T_{(j+1)}).$$

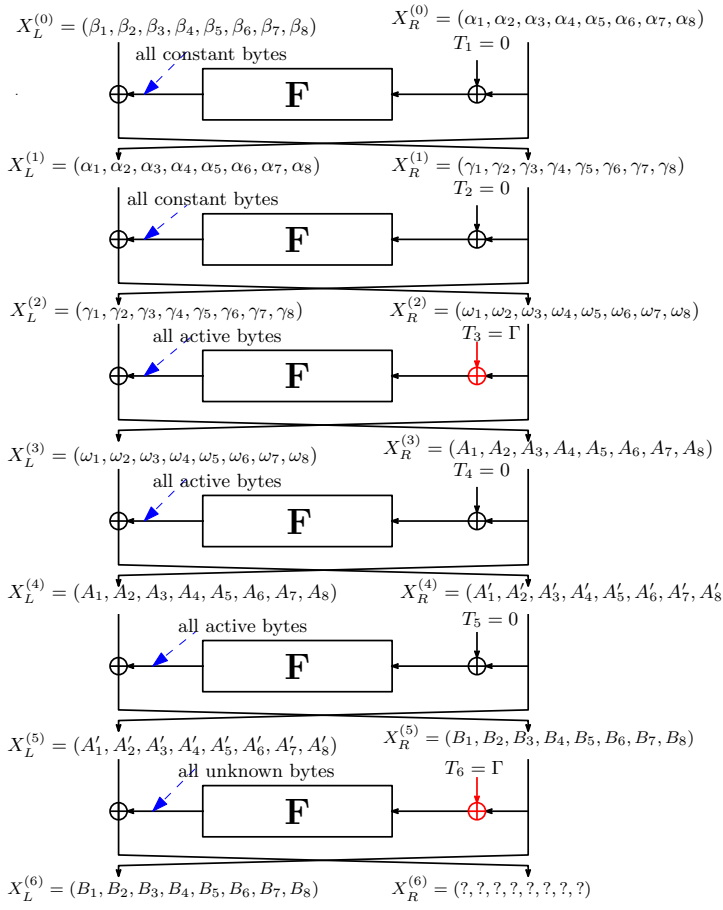

Fig. 6. A 6-round distinguisher for FEA-2.

4. Check that all bytes of $X_L^{(6)}$ are balanced.

We further describe the details of the six-round distinguisher attack procedure. Start with choosing plaintexts $X_L^{(0)}$ and $X_R^{(0)}$ as follows:

$$X_L^{(0)} = (\beta_1, \beta_2, \beta_3, \beta_4, \beta_5, \beta_6, \beta_7, \beta_8) \tag{19}$$

$$X_R^{(0)} = (\alpha_1, \alpha_2, \alpha_3, \alpha_4, \alpha_5, \alpha_6, \alpha_7, \alpha_8), \tag{20}$$

where each byte β_i for $1 \leq i \leq 8$, and each α_i for $1 \leq i \leq 8$ are arbitrary constants, i.e., all plaintext bytes are constants.

After executing the first round on inputs $(X_L^{(0)}, X_R^{(0)})$, the second round inputs $X_L^{(1)}$ and $X_R^{(1)}$ become:

$$X_L^{(1)} = (\alpha_1, \alpha_2, \alpha_3, \alpha_4, \alpha_5, \alpha_6, \alpha_7, \alpha_8) \tag{21}$$

$$X_R^{(1)} = (\gamma_1, \gamma_2, \gamma_3, \gamma_4, \gamma_5, \gamma_6, \gamma_7, \gamma_8). \tag{22}$$

After executing the second round on inputs $(X_L^{(1)}, X_R^{(1)})$, the third round inputs $X_L^{(2)}$ and $X_R^{(2)}$ become:

$$X_L^{(2)} = (\gamma_1, \gamma_2, \gamma_3, \gamma_4, \gamma_5, \gamma_6, \gamma_7, \gamma_8) \tag{23}$$

$$X_R^{(2)} = (\omega_1, \omega_2, \omega_3, \omega_4, \omega_5, \omega_6, \omega_7, \omega_8). \tag{24}$$

For third round, the tweak T_3 forms a Γ set. The input and output of round functions are γ sets. After executing the third round on inputs $(X_L^{(2)}, X_R^{(2)})$, the third round inputs $X_L^{(3)}$ and $X_R^{(3)}$ become:

$$X_L^{(3)} = (\omega_1, \omega_2, \omega_3, \omega_4, \omega_5, \omega_6, \omega_7, \omega_8) \tag{25}$$

$$X_R^{(3)} = (A_1, A_2, A_3, A_4, A_5, A_6, A_7, A_8). \tag{26}$$

After executing the forth round on inputs $(X_L^{(3)}, X_R^{(3)})$, the fifth round inputs $X_L^{(4)}$ and $X_R^{(4)}$ become:

$$X_L^{(4)} = (A_1, A_2, A_3, A_4, A_5, A_6, A_7, A_8) \tag{27}$$

$$X_R^{(4)} = (A'_1, A'_2, A'_3, A'_4, A'_5, A'_6, A'_7, A'_8). \tag{28}$$

where each A'_i forms a Λ-set consisting all possible values from $\{0,1\}^8$.

After executing the fifth round on inputs $(X_L^{(4)}, X_R^{(4)})$, the sixth round inputs $X_L^{(5)}$ and $X_R^{(5)}$ become:

$$X_L^{(5)} = (A'_1, A'_2, A'_3, A'_4, A'_5, A'_6, A'_7, A'_8) \tag{29}$$

$$X_R^{(5)} = (B_1, B_2, B_3, B_4, B_5, B_6, B_7, B_8). \tag{30}$$

After executing the sixth round on inputs $(X_L^{(5)}, X_R^{(5)})$, the seventh round inputs $X_L^{(6)}$ and $X_R^{(6)}$ become:

$$X_L^{(6)} = (B_1, B_2, B_3, B_4, B_5, B_6, B_7, B_8) \tag{31}$$

$$X_R^{(6)} = (?, ?, ?, ?, ?, ?, ?, ?). \tag{32}$$

As expected, each byte of right-half output of $X_R^{(6)}$ are balanced (refer to Fig. 6). In order to mount this distinguisher, we need 2^{64} number of six round encryptions. Consequently, the time and data complexity of this distinguisher are both 2^{64} plaintexts and encryptions, respectively.

5 Key Recovery Attacks

In this section, we describe key recovery attacks on FEA-1 and FEA-2 using the distinguishers presented in the Subsections 4.1 and 4.2, respectively. Firstly, we describe five round key recovery attack for FEA-1, using the four round distinguisher. Afterwards, we describe seven round key recovery attack for FEA-2, using the six round distinguisher. Both of these attacks are applicable only to the 256-bit key sizes of FEA-1 and FEA-2.

5.1 Five Round Key Recovery Attacks on FEA-1

In order to construct this five round key recovery attack on FEA-1, we extend the four round distinguisher presented in Subsection 4.1 by adding an extra round at the end. The attack is demonstrated in Fig. 7.

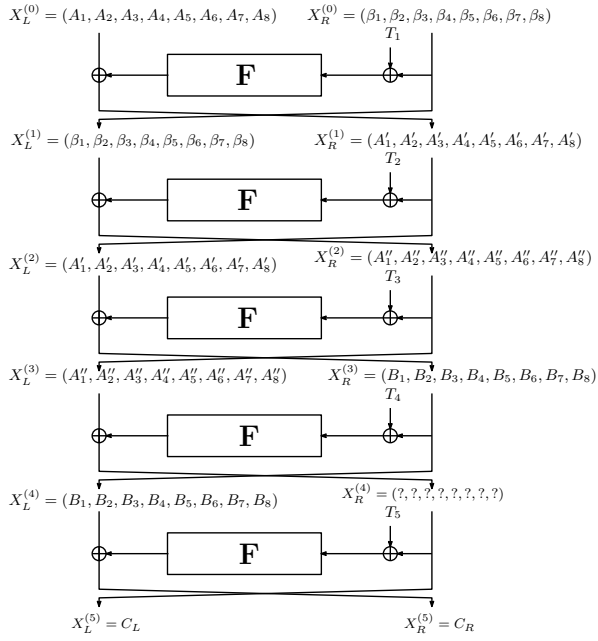

Fig. 7. 5-round key recovery attack of FEA-1.

This five round attack begins by choosing a collection of plaintexts such that the first round input forms a Γ-set, as described in Equations (11) and (12). Note that the tweak value T is always 0 (by design specification of FEA-1). After choosing plaintexts, we run five rounds of FEA-1. Let C_L, C_R represents the left and right halves of five round output, respectively, i.e., $X_L^{(5)} = C_L$ and $X_R^{(5)} = C_R$.

To check the balance property at the end of the forth round, we decrypt the ciphertext C_L, C_R for one round by guessing all the 16-bytes of round key (eight bytes of each $Rk_a^{(5)}$ and $Rk_b^{(5)}$) for all 2^{64} ciphertexts set. After decryption, check whether XOR all 2^{64} values $X_L^{(4)}$ is balanced or not. If $X_L^{(4)}$ is not balanced, then the guessed round key is a wrong key. Otherwise, the guessed key is likely the correct key.

A wrong key can pass this test with a probability 2^{-64}. To filter out the wrong keys, we need three Γ-sets of plaintexts as inputs. We require 2^{64} number of five round encryptions for each guessed key. Therefore, the time complexity of five round key-recovery attack is $3 \times 2^{64} \times (2^8)^{16} \approx 2^{193.6}$.

5.2 Seven Round Key Recovery Attack on FEA-2

In order to construct this seven round key recovery attack on FEA-2, we extend the six round distinguisher presented in Subsection 4.2 by adding an extra round at the end. The attack is depicted in Fig. 8.

This seven round attack begins by choosing a collection of plaintexts such that the third round input forms a Γ-set, as described in Equations (25) and (26). It is important to note that the tweak value T_1 is always 0 (by design specification of FEA-2), while we set $T_2 = T_4 = T_5 = 0$ and choose T_3 such that $(T_3 \oplus X_R^{(2)})$ forms a Γ-set. Basically, here we choose round tweak T_3 a Γ-set. After choosing plaintexts and round tweaks, we run seven rounds of FEA-2. Let C_L, C_R represents the left and right halves of seventh round output, respectively, i.e., $X_L^{(7)} = C_L$ and $X_R^{(7)} = C_R$.

To check the balance property at the end of the sixth round, we decrypt the ciphertext C_L, C_R for one round by guessing all the 16-bytes of round key of seventh round, for all 2^{64} ciphertexts of the set. After decrypting, check whether the XOR of all 2^{64} values of $x_L^{(6)}$ equals zero, i.e., $X_L^{(6)}$ is balanced. If $X_L^{(6)}$ is not balanced, then the guessed round key is a wrong key. Otherwise, the guessed key is likely the correct key.

A wrong key can pass this test with a probability 2^{-64}. To filter out the wrong keys, we need three Γ-sets of plaintexts as inputs. We require 2^{64} number of seven round encryptions for each guessed key. Therefore, the time complexity of seven round key-recovery attack is $3 \times 2^{64} \times (2^8)^{16} \approx 2^{193.6}$.

6 Conclusion and Future Works

We presented two new distinguishers for FEA-1 and FEA-2 that cover more rounds than existing works, and extended these distinguishers to mount key-recovery attacks. Our analysis emphasizes that both reduced-round FEA-1 and FEA-2 is vulnerable to square cryptanalysis beyond the designer's claim. The limitation of the proposed attacks is that it is useful only for those cases where the input size is 128-bit. Security analysis of FEA-1 and FEA-2 against other cryptanalytic techniques is an interesting line of future work.

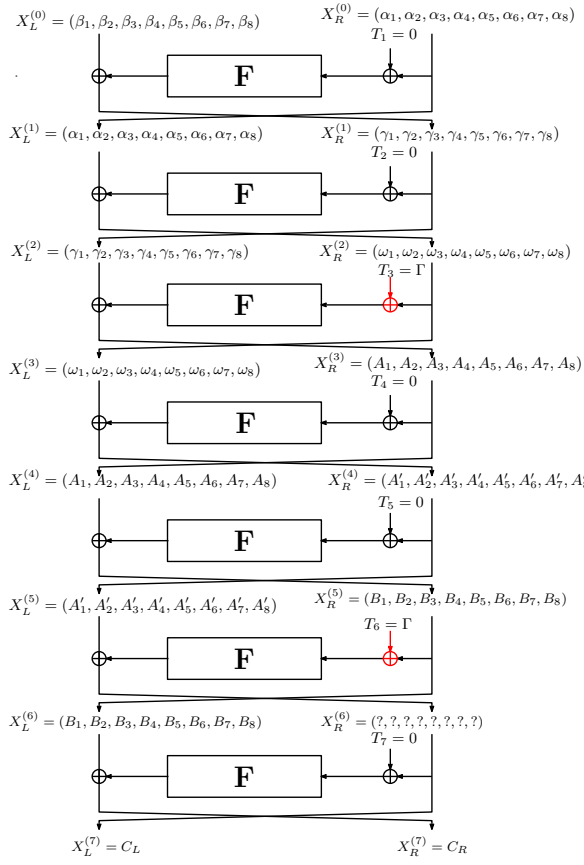

Fig. 8. 7-round key recovery attack of FEA-2.

References

1. Mihir Bellare, Thomas Ristenpart, Phillip Rogaway, and Till Stegers. Format-preserving encryption. In *Selected Areas in Cryptography, 2009*, volume 5867 of *Lecture Notes in Computer Science*, pages 295–312. Springer, 2009.
2. Tim Beyne. Linear cryptanalysis of FF3-1 and FEA. In *Advances in Cryptology - CRYPTO 2021*, volume 12825 of *Lecture Notes in Computer Science*, pages 41–69. Springer, 2021.
3. John Black and Phillip Rogaway. Ciphers with arbitrary finite domains. In *Topics in Cryptology - CT-RSA 2002,*, volume 2271 of *Lecture Notes in Computer Science*, pages 114–130. Springer, 2002.
4. Andrey Bogdanov, Lars R. Knudsen, Gregor Leander, Christof Paar, Axel Poschmann, Matthew J. B. Robshaw, Yannick Seurin, and C. Vikkelsoe. PRESENT: an ultra-lightweight block cipher. In Pascal Paillier and Ingrid Verbauwhede, editors, *Cryptographic Hardware and Embedded Systems - CHES 2007, 9th International Workshop, Vienna, Austria, September 10-13, 2007,*

Proceedings, volume 4727 of *Lecture Notes in Computer Science*, pages 450–466. Springer, 2007.

5. Andrey Bogdanov and Kyoji Shibutani. Double sp-functions: enhanced generalized feistel networks. In *Information Security and Privacy: 16th Australasian Conference, ACISP 2011, Melbourne, Australia, July 11-13, 2011. Proceedings 16*, pages 106–119. Springer, 2011.
6. Eric Brier, Thomas Peyrin, and Jacques Stern. BPS: a format-preserving encryption proposal. *Submission to NIST*, 2010.
7. Michael Brightwell and H Smith. Using datatype-preserving encryption to enhance data warehouse security. In *20th National Information Systems Security Conference Proceedings (NISSC)*, pages 141–149, 1997.
8. Donghoon Chang, Mohona Ghosh, Kishan Chand Gupta, Arpan Jati, Abhishek Kumar, Dukjae Moon, Indranil Ghosh Ray, and Somitra Kumar Sanadhya. SPF: A new family of efficient format-preserving encryption algorithms. In *Information Security and Cryptology, Inscrypt 2016*, volume 10143 of *Lecture Notes in Computer Science*, pages 64–83. Springer, 2016.
9. Donghoon Chang, Mohona Ghosh, Arpan Jati, Abhishek Kumar, and Somitra Kumar Sanadhya. A generalized format preserving encryption framework using MDS matrices. *J. Hardw. Syst. Secur.*, 3(1):3–11, 2019.
10. Amit Kumar Chauhan, Abhishek Kumar, and Somitra Kumar Sanadhya. Square attacks on reduced-round FEA-1 and FEA-2. In *Stabilization, Safety, and Security of Distributed Systems - 25th International Symposium, SSS 2023, Jersey City, NJ, USA, October 2023*, volume 14310 of *Lecture Notes in Computer Science*, pages 583–597. Springer, 2023.
11. Joan Daemen, Lars R. Knudsen, and Vincent Rijmen. The block cipher square. In *Fast Software Encryption, 4th International Workshop, FSE '97*, volume 1267 of *Lecture Notes in Computer Science*, pages 149–165. Springer, 1997.
12. Joan Daemen and Vincent Rijmen. *The Design of Rijndael: AES - The Advanced Encryption Standard*. Information Security and Cryptography. Springer, 2002.
13. Christoph Dobraunig, Maria Eichlseder, and Florian Mendel. Square attack on 7-round kiasu-bc. In *Applied Cryptography and Network Security: 14th International Conference, ACNS 2016, Guildford, UK, June 19-22, 2016. Proceedings 14*, pages 500–517. Springer, 2016.
14. Orr Dunkelman, Abhishek Kumar, Eran Lambooij, and Somitra Kumar Sanadhya. Cryptanalysis of feistel-based format-preserving encryption. *IACR Cryptol. ePrint Arch.*, page 1311, 2020.
15. F. Betül Durak, Henning Horst, Michael Horst, and Serge Vaudenay. FAST: secure and high performance format-preserving encryption and tokenization. In *Advances in Cryptology - ASIACRYPT*, volume 13092, pages 465–489. Springer, 2021.
16. M. Dworkin. NIST Special Publication 800-38A: Recommendation for Block Cipher Modes of Operation-Methods and Techniques, 2001.
17. Morris Dworkin. Recommendation for block cipher modes of operation: Methods for format-preserving encryption. *NIST Special Publication SP 800-38G Rev. 1*, 800-38G, 2019.
18. Horst Feistel. Block cipher cryptographic system, March 19 1974. US Patent 3,798,359.
19. Louis Granboulan, Éric Levieil, and Gilles Piret. Pseudorandom permutation families over abelian groups. In *Fast Software Encryption, FSE 2006*, volume 4047 of *Lecture Notes in Computer Science*, pages 57–77. Springer, 2006.

20. Shay Gueron and Nicky Mouha. Simpira v2: A family of efficient permutations using the aes round function. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 95–125. Springer, 2016.
21. Jung-Keun Lee, Bonwook Koo, Dongyoung Roh, Woo-Hwan Kim, and Daesung Kwon. Format-preserving encryption algorithms using families of tweakable blockciphers. In *Information Security and Cryptology - ICISC*, volume 8949, pages 132–159. Springer, 2014.
22. Stefan Lucks. Faster luby-rackoff ciphers. In *International Workshop on Fast Software Encryption*, pages 189–203. Springer, 1996.
23. Mitsuru Matsui. Linear cryptanalysis method for des cipher. In *Workshop on the Theory and Application of of Cryptographic Techniques*, pages 386–397. Springer, 1993.
24. T. Spies. Feistel Finite Set Encryption. NIST submission, February 2008, Available at `http://csrc.nist.gov/groups/ST/toolkit/BCM/modes-development.html`.
25. Yongjin Yeom, Sangwoo Park, and Iljun Kim. On the security of CAMELLIA against the square attack. In *Fast Software Encryption, 9th International Workshop, FSE 2002*, volume 2365 of *Lecture Notes in Computer Science*, pages 89–99. Springer, 2002.